

Dory.md — Person 2

Intelligence Engine

Decay model · BGE embeddings · BM25 + RRF hybrid search · Groq classifier · Quiz generator · Demo dataset

Your role	Intelligence Engineer — you own everything in /intelligence/. You do NOT build a server, you build Python modules that Person 1 imports and calls.
What you ship	Six modules: decay.py (NumPy retention math), embedder.py (BGE + ChromaDB), search.py (BM25 + dense + RRF), classifier.py (Groq), quiz.py (Groq MCQs), demo_data.py (backdated dataset generator for the demo).
What you DON'T do	No FastAPI code. No SQLite schema. No frontend code. No deployment work. If a prompt needs any of those, the request belongs to Person 1 or Person 3.
Your Hour 0-48 path	Setup + demo data generator (Hours 0-4) → Decay engine (Hours 4-8) → Embedder + ChromaDB (Hours 8-14) → Hybrid search (Hours 14-20) → Groq classifier (Hours 20-24) → Quiz generator (Hours 24-30) → Integration testing with P1 (Hours 30-38) → Polish + buffer (Hours 38-48).
Riskiest task	First-time model download. sentence_transformers downloads ~90MB on first use; if the venue WiFi is bad this takes 15+ minutes. Do this BEFORE the hackathon starts. The second risk is Groq rate limits during the demo if you call it synchronously per chunk; the doc shows you how to batch.
Branch strategy	Work on branch feat/intelligence. Person 1 will pull your branch around Hour 18-26 to wire your modules into their endpoints. Keep public APIs stable from Hour 8 onward — internal implementation can change, signatures cannot.

Contents

- 1. Before You Start — Checklist
- 2. The Three-Person Integration Contract
- 3. Pre-Hackathon Setup (Do This DAYS Before)
- 4. Hours 0-4 — Project Setup & Demo Dataset Generator
- 5. Hours 4-8 — Decay Engine (the Time Machine math)
- 6. Hours 8-14 — Embedder + ChromaDB
- 7. Hours 14-20 — Hybrid Search (BM25 + Dense + RRF)
- 8. Hours 20-24 — Groq Classifier
- 9. Hours 24-30 — Quiz Generator
- 10. Hours 30-38 — Integration Testing with Person 1
- 11. Hours 38-48 — Polish, Bug Bash, Demo Prep
- 12. Stretch Tasks (if you finish early)
- 13. Descope Ladder (if you fall behind)
- 14. Handoff Checklists
- 15. Troubleshooting Guide

1. Before You Start — Checklist

If any of these isn't true, fix it before the hackathon starts. The model download in particular cannot be done at the venue if WiFi is flaky.

✓ / ✗	Item	How to verify
	Python 3.11+ installed	Terminal: <code>python --version</code>
	Git installed and configured	Terminal: <code>git config --list</code>
	GitHub access to team repo	Try git push to a test branch
	Code editor with LLM (Cursor/Copilot/etc.)	Open project, paste a test prompt
	Groq API key (the SAME key Person 1 uses)	Test: <code>curl https://api.groq.com/openai/v1/models -H 'Authorization: Bearer \$GROQ_API_KEY'</code>
	Pre-downloaded BGE-small model	See Section 3.1 — this is critical
	Cloned the team repo locally	<code>git clone <repo></code>
	Created your branch	<code>git checkout -b feat/intelligence</code>
	Read this entire document end to end	Don't write code until you have
	Aligned on the JSON contract with Person 1	See Section 2 — verify Chunk schema matches

2. The Three-Person Integration Contract

The Three-Person Integration Contract

All three of you must agree on these JSON shapes BEFORE any of you start building. If any field changes during the hackathon, the person changing it must notify the other two in Slack/Discord immediately. The contract is the single source of truth — everything else flows from it.

Endpoint	Method	Owner	Returns
/api/health	GET	P1 routes, P2 logic	{categories: [{name, retention, urgency, count}]}
/api/health?time_offset_hours=N	GET	P1 routes, P2 logic	Same shape, computed at t+N
/api/ingest	POST	P1 fully	{status, chunks_count, doc_id}
/api/search	POST	P1 routes, P2 logic	{results: [{chunk_id, content, score, retention}]}
/api/discovery	GET	P1 routes, P2 logic	{has_discovery, chunk?, reason?}
/api/review/{chunk_id}	POST	P1 fully	{status, new_access_count}
/api/quiz/start	POST	P1 routes, P2 logic	{quiz_id, questions: [{q, options, correct}]}
/api/quiz/submit	POST	P1 fully	{score, correct_count}

The Chunk JSON Shape

This is the canonical shape of a Chunk object. Person 1 stores it, Person 2 enriches and retrieves it, Person 3 displays it. If you need a field not listed here, propose the addition in chat before adding it.

```
{ "id": "chk_8a3f...", // UUID, generated by P1
  "content": "...chunk text 200-500 chars...", // P1 splits source
  "source_type": "file", // P1 sets at ingest
  "source_name": "research_notes.md", // filename
  "category": "technical" | "personal" | ..., // P2 sets via Groq
  "created_at": "2026-04-25T10:00:00Z", // ISO 8601
  "last_accessed": "2026-04-25T10:00:00Z", // updated on review
  "access_count": 0, // increments on review
  "stability_S": 7.0, // P2 decay parameter
  "complexity_k": 1.0 // P2 decay parameter
}
```

Your contract with Person 1 (Backend)

Person 1 imports your modules. They will call your functions with these exact signatures. **If you change a signature, tell P1 immediately.**

Your file	Function signature	When P1 calls it
intelligence/decay.py	compute_retention(chunks: list[Chunk], time_offset_hours: float = 0) -> np.ndarray	Every /api/health request
intelligence/decay.py	aggregate_by_category(chunks: list[Chunk], retentions: np.ndarray) -> dict	Every /api/health request
intelligence/embedder.py	embed_chunks(texts: list[str]) -> np.ndarray	Background task after every ingest
intelligence/embedder.py	store_embeddings(chunks: list[Chunk], embeddings: np.ndarray) -> None	Background task after every ingest
intelligence/search.py	hybrid_search(query: str, top_k: int = 10) -> list[tuple[str, float]]	Every /api/search request
intelligence/classifier.py	classify_chunks(texts: list[str]) -> list[str]	Background task after every ingest
intelligence/quiz.py	generate_quiz_questions(chunks: list[Chunk]) -> list[dict]	Every /api/quiz/start request

Your contract with Person 3 (Frontend)

You don't talk to Person 3 directly — only through Person 1's API. But you should know what they expect, because mistakes in your output show up in their UI:

- Categories must be one of: technical, personal, reference, general. Person 3 has CSS classes for exactly these four.
- Retention values must be in [0, 1]. Anything outside breaks the dashboard color scale.
- Quiz questions must have exactly 4 options each. Their UI assumes 4 buttons.
- Search results must include retention field for each result so P3 can show the "forgetting" badge.

3. Pre-Hackathon Setup (Do This DAYS Before)

3.1 — Pre-download the BGE-small model

STOP — HUMAN STEP REQUIRED

This is the single most important pre-hackathon task for you. Do it on stable WiFi at home, NOT at the venue.

Why: sentence_transformers downloads the model from Hugging Face the first time you use it. The BGE-small model is ~133MB. On the venue's congested WiFi, this can take 15 minutes or fail. Once downloaded, it's cached at ~/.cache/huggingface/ and subsequent loads take seconds.

Steps:

1. Create a Python venv: `python -m venv .venv && source .venv/bin/activate`
2. Install: `pip install sentence-transformers torch numpy`
3. Run this Python snippet ONCE on your home WiFi:

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('BAAI/bge-small-en-v1.5')
print('Downloaded. Test embed:', model.encode('hello world').shape)
```
4. Verify the cache exists: `ls -lh ~/.cache/huggingface/hub/` should show the model files
5. **Never delete this cache before the hackathon.** If you reformat your laptop, redo this step.

3.2 — Pre-install all Python dependencies

STOP — HUMAN STEP REQUIRED

Steps:

1. With your venv active, install everything you'll need:
`pip install sentence-transformers chromadb rank-bm25 numpy pandas groq python-dotenv pytest pytest-asyncio`
2. Verify each imports cleanly: `python -c 'import sentence_transformers, chromadb, rank_bm25, groq; print("OK")'`
3. **chromadb specifically** downloads its own ONNX model the first time. Force it:
`python -c 'import chromadb; c = chromadb.Client(); c.create_collection("test")'`
4. If anything fails to install, fix it now — not at the venue.

3.3 — Verify Groq access

STOP — HUMAN STEP REQUIRED

Steps:

1. Make sure GROQ_API_KEY is in your .env (same key Person 1 has)
2. Test it: `curl https://api.groq.com/openai/v1/models -H "Authorization: Bearer $GROQ_API_KEY"`
3. You should see a JSON list of available models including llama-3.3-70b-versatile
4. If you get 401 Unauthorized, the key is wrong or expired — get a new one from console.groq.com
5. Note the rate limits: ~30 requests/min on free tier. This matters for batching strategy.

3.4 — Sync with Person 1 on schema

Before the hackathon, message Person 1 with: *'Confirming the Chunk schema in our contract — I'll write code that expects these fields exactly: id, content, source_type, source_name, category, created_at, last_accessed, access_count, stability_S, complexity_k. If anything changes, ping me before merging.'*

4. Hours 0-4 — Project Setup & Demo Dataset Generator

Goal: project structure ready, a demo dataset generator that produces realistic backdated data for the Time Machine slider to work on Day 0.

Strategy: while Person 1 is still building the database, you can work entirely offline against the agreed Chunk schema. The demo dataset generator is your first deliverable — it's also a stretch feature you flagged earlier as pending.

Prompt 1 — Set up the intelligence package structure

PROMPT TO PASTE INTO YOUR LLM

Inside the existing dory-md/ repository, set up the intelligence/ package. The folder already exists with an empty `__init__.py` from Person 1.

Create this structure inside intelligence/:

```
intelligence/
■■■ __init__.py # Package marker
■■■ decay.py # NumPy retention math
■■■ embedder.py # BGE + ChromaDB
■■■ search.py # BM25 + dense + RRF
■■■ classifier.py # Groq categorization
■■■ quiz.py # Groq MCQ generation
■■■ demo_data.py # Backdated dataset generator
■■■ chunk_protocol.py # Type protocol for Chunk objects
■■■ tests/
■■■ __init__.py
```

For now create empty stub files for each module with a docstring describing its purpose. In `chunk_protocol.py`, define a `typing.Protocol` class called `ChunkLike` with the fields from the contract (`id`, `content`, `source_type`, `source_name`, `category`, `created_at`, `last_accessed`, `access_count`, `stability_S`, `complexity_k`). Use this Protocol everywhere your code needs to type-hint a chunk-shaped object — this keeps your modules decoupled from Person 1's SQLAlchemy models.

Do not write actual logic yet. Just structure + the Protocol.

Prompt 2 — Demo dataset generator

PROMPT TO PASTE INTO YOUR LLM

In `intelligence/demo_data.py`, write a generator that produces a realistic demo dataset for the Time Machine slider. The dataset must have:

1. ~80-100 chunks distributed across 4 categories (technical, personal, reference, general)
2. Realistic content — short paragraphs about plausible topics for each category
3. Backdated timestamps spanning the past 60 days (some chunks 60 days old, some 30, some 7, some 1)
4. Varied `access_count` values (some chunks accessed many times, some never)
5. Varied `stability_S` values (some at default 7.0, some boosted up to 100 from reviews)

The function signature:

```
def generate_demo_chunks(count: int = 90, seed: int = 42) -> list[dict]
```

Returns a list of dicts matching the Chunk schema. Use a seed so the demo is reproducible.

Content templates per category (use these as seeds, vary by topic):

- technical: 'Notes on async patterns in Python', 'PostgreSQL performance tuning', 'React useEffect cleanup', 'Docker volume mounting', 'TypeScript generics'

- personal: 'Coffee shop in Capitol Hill I want to revisit', 'Recipe ideas', 'Books to read this quarter', 'Workout routine', 'Travel plans for spring'
- reference: 'NumPy broadcasting rules', 'HTTP status code meanings', 'Git commands I always forget', 'Regex cheat sheet'
- general: 'Meeting notes', 'Random thoughts', 'Half-formed ideas', 'Article summaries'

For each chunk, generate:

- id: `'chk_' + uuid.uuid4().hex[:12]`
- content: 200-400 chars of realistic text. Use a small library of templates and string formatting to vary them.
- category: weighted random – 35% technical, 25% personal, 15% reference, 25% general
- created_at: random datetime in the past 60 days
- last_accessed: usually equal to created_at, but for ~30% of chunks set it more recent (simulating that they've been re-read)
- access_count: 0 for most, 1-5 for the recently-accessed 30%
- stability_S: 7.0 if access_count is 0, otherwise $7.0 * (1.5 ** \text{access_count})$ capped at 365
- complexity_k: 1.0 for general/personal, 0.7 for technical/reference (technical decays faster)

Also add a function:

```
def write_demo_dataset_to_file(path: str = 'demo_chunks.json', count: int = 90)
Which calls generate_demo_chunks and writes the JSON to a file with proper datetime
serialization.
```

After implementation, write a test in intelligence/tests/test_demo_data.py that:

1. Generates 90 chunks and verifies the count
2. Verifies all required fields are present
3. Verifies retention math will work – i.e., for an old chunk with access_count=0, S=7.0, k=1.0, the formula $\exp(-\text{hours_since_access} / (S * k * 24))$ should give a value noticeably below 1.0

Prompt 3 — Generate the demo file and inspect it

PROMPT TO PASTE INTO YOUR LLM

Run the demo data generator to produce demo_chunks.json. Then write a small inspection script intelligence/tests/inspect_demo.py that:

1. Loads the JSON
2. Prints the count of chunks per category
3. Prints the date range (oldest and newest created_at)
4. Prints 3 random sample chunks for me to eyeball quality
5. Prints the distribution of access_count values

Run the script and paste the output back to me so I can verify the dataset looks realistic. If chunks look generic or repetitive, we'll improve the templates.

Heads-up

Person 1's /api/ingest endpoint takes file uploads, not JSON dicts directly. So you'll need a small loader script that reads demo_chunks.json and inserts the chunks DIRECTLY into the SQLite database, bypassing the ingest endpoint. We'll write that around Hour 38 during demo prep. For now, your demo data lives as a JSON file.

Commit checkpoint

git add -A && git commit -m 'feat(intelligence): demo dataset generator + package scaffolding' then push.

5. Hours 4-8 — Decay Engine (the Time Machine math)

Goal: a fast NumPy-vectorized retention computation that powers the dashboard and the Time Machine slider. Target: under 1ms for 2,300 chunks.

The math (from your blueprint)

Retention $R(t) = \exp(-\text{hours_since_access} / (S \times k \times 24))$ where:

- $\text{hours_since_access} = (\text{now} - \text{last_accessed})$ in hours, plus optional time_offset_hours for the slider
- $S = \text{stability_S}$ (chunk attribute, increases with reviews)
- $k = \text{complexity_k}$ (chunk attribute, complexity modifier)
- Multiplying by 24 converts S (in days) to hours

Prompt 4 — The vectorized decay function

PROMPT TO PASTE INTO YOUR LLM

In intelligence/decay.py, implement the core retention computation using NumPy.

Function signature:

```
def compute_retention(
    chunks: list, # List of ChunkLike objects
    time_offset_hours: float = 0
) -> np.ndarray
```

Returns: np.ndarray of shape (len(chunks),) with values in [0, 1].

Implementation:

1. Build NumPy arrays from chunks (do this once, vectorize the rest):
 - $\text{last_accessed_arr} = \text{array of last_accessed datetimes}$
 - $S_arr = \text{array of stability_S values}$
 - $k_arr = \text{array of complexity_k values}$
2. Compute $\text{hours_since_access} = ((\text{now} - \text{last_accessed_arr}).\text{total_seconds}() / 3600) + \text{time_offset_hours}$. Use np.array of pd.Timestamp differences, or convert to Unix timestamps before subtraction for speed.
3. Compute $\text{retentions} = \text{np.exp}(-\text{hours_since_access} / (S_arr * k_arr * 24))$
4. Clamp to [0, 1] using np.clip
5. Handle edge cases:
 - Empty chunks list → return np.array([])
 - Negative time_offset_hours (user dragged slider backward) → still works mathematically, just gives high retention
 - Chunks where last_accessed is in the future → treat as 1.0 (just-accessed)

Performance target: under 1ms for 2,300 chunks. Use timeit to benchmark.

Add a benchmark function:

```
def benchmark_decay(n: int = 2300):
    Creates n synthetic chunks, runs compute_retention 100 times, prints avg ms per call.
```

Run the benchmark and paste the output. If it's over 1ms, we'll optimize.

Prompt 5 — Aggregate by category

PROMPT TO PASTE INTO YOUR LLM

In intelligence/decay.py, add the aggregation function for the dashboard.

```
def aggregate_by_category(
    chunks: list,
    retentions: np.ndarray
) -> dict
```

Returns:

```
{
    'categories': [
        {
            'name': str,
            'avg_retention': float, # mean retention for chunks in this category
            'min_retention': float, # lowest retention (most-forgotten chunk)
            'count': int, # number of chunks in this category
            'urgency': str # 'low' | 'medium' | 'high'
        },
        ...
    ],
    'total_chunks': int,
    'overall_retention': float # mean across all chunks
}
```

Urgency mapping based on avg_retention:

- $\geq 0.7 \rightarrow$ 'low'
- 0.4 to 0.7 $\rightarrow$ 'medium'
- $< 0.4 \rightarrow$ 'high'

Implementation: use pandas groupby for cleanness, or pure NumPy with np.unique + boolean masking. Both work; benchmark both and use the faster one.

Edge cases:

- Empty chunks list $\rightarrow$ return {'categories': [], 'total_chunks': 0, 'overall_retention': 0.0}
- A category with 0 chunks should not appear in the output

Add a unit test that creates 10 fake chunks across 3 categories, computes retention, calls aggregate_by_category, and verifies the counts and urgency labels are correct.

Prompt 6 — Test the Time Machine slider math

PROMPT TO PASTE INTO YOUR LLM

Write a script `intelligence/tests/test_time_machine.py` that simulates the Time Machine slider behavior:

1. Load the `demo_chunks.json` (from Hour 0-4)
2. For `time_offset_hours` in `[0, 24, 168, 720, 2160]` (now, +1 day, +1 week, +1 month, +3 months):
 - a. Compute retentions
 - b. Aggregate by category
 - c. Print: 'At t+{offset}h: technical avg={x}, personal avg={y}, ...'
3. Verify that retentions decrease monotonically as `time_offset_hours` increases
4. Verify that no retention value goes outside `[0, 1]`

Run the script and paste the output. The output is the data the Time Machine slider will show on the demo. If categories don't decay at sensible rates, we'll tune.

Prompt 7 — Optimize if needed

PROMPT TO PASTE INTO YOUR LLM

If `benchmark_decay` shows over 1ms for 2,300 chunks, optimize:

1. Pre-convert chunk datetimes to Unix timestamps once at module level (cached) instead of per-call
2. Use `float32` instead of `float64` in the NumPy arrays
3. Profile with `cProfile` to find the actual bottleneck

If under 1ms, skip optimization and move on. Optimization without measurement is wasted effort during a hackathon.

Commit checkpoint

`git commit -m 'feat(intelligence): vectorized decay engine'`. At Hour 8 your decay module is the most-tested code in the repo and powers the demo's signature feature.

6. Hours 8-14 — Embedder + ChromaDB

Goal: BGE-small embedding pipeline that stores vectors in ChromaDB and supports semantic search by query.

Prompt 8 — Wrap BGE-small with caching

PROMPT TO PASTE INTO YOUR LLM

In intelligence/embedder.py, build the embedding wrapper.

Module-level setup:

1. Load the SentenceTransformer model ONCE at module import:
`<code>_MODEL = SentenceTransformer('BAAI/bge-small-en-v1.5')</code>`
2. Set `<code>_MODEL.max_seq_length = 256</code>` (chunks are short, this speeds inference)
3. Define the query instruction prefix used by BGE:
`<code>QUERY_INSTRUCTION = 'Represent this sentence for searching relevant passages: '</code>`

Functions:

```
def embed_chunks(texts: list[str]) -> np.ndarray
Encodes a list of chunk texts. Returns shape (len(texts), 384). Use
batch_size=32, normalize_embeddings=True. NO instruction prefix (only queries
get it).
Edge case: empty list → return np.zeros((0, 384))

def embed_query(query: str) -> np.ndarray
Encodes a single query string with the instruction prefix. Returns shape (384,).
Use normalize_embeddings=True.

def cosine_similarity_batch(
    query_emb: np.ndarray, # shape (384,)
    chunk_embs: np.ndarray # shape (N, 384)
) -> np.ndarray # shape (N,)
Computes cosine similarity. Since embeddings are L2-normalized, this is just dot
product: <code>chunk_embs @ query_emb</code>. Returns float values in [-1, 1].
```

Add a benchmark:

```
def benchmark_embed(n: int = 100):
Generates n random 200-char strings, calls embed_chunks, prints throughput.
```

Run benchmark, expected throughput is around 50-200 chunks/sec on CPU. If it's much slower, the model didn't load right.

Heads-up

First call to SentenceTransformer('BAAI/bge-small-en-v1.5') will be slow (2-15 seconds) even after the model is cached, because it has to load weights into RAM. Subsequent imports are fast. Don't put the model load inside the embed_chunks function — keep it at module level so it loads once.

Prompt 9 — ChromaDB integration

PROMPT TO PASTE INTO YOUR LLM

In intelligence/embedder.py, add ChromaDB persistence.

Module-level setup:

1. Use chromadb.PersistentClient pointing at './chroma_db' (gitignore this folder)

2. Create or get a collection called 'dory_chunks' with these settings:

```
<code>configuration={'hnsw': {'space': 'cosine', 'M': 16, 'ef_construction': 200, 'ef_search': 50}}</code>
```

3. CRITICAL: M and ef_construction can NEVER be changed after creation. If the collection already exists with different settings, log a warning and use the existing collection.

Functions:

```
def store_embeddings(
    chunks: list, # ChunkLike objects with .id and .content
    embeddings: np.ndarray # shape (len(chunks), 384) – pre-computed
) -> None
Adds to ChromaDB:
• ids = [c.id for c in chunks]
• embeddings = embeddings.tolist()
• documents = [c.content for c in chunks]
• metadatas = [{'category': c.category, 'source_type': c.source_type, 'source_name':
c.source_name} for c in chunks]
Use collection.add(). If a chunk ID already exists, use upsert=True semantics – ChromaDB has
collection.upsert() for this.

def query_embeddings(
    query_emb: np.ndarray, # shape (384,)
    top_k: int = 50
) -> list[tuple[str, float, str]]
Returns list of (chunk_id, similarity, content) tuples, sorted by similarity desc.
ChromaDB returns 'distances' which is 1 - cosine_similarity for cosine space.
So: similarity = 1 - distance. Convert before returning.

def get_collection_stats() -> dict
Returns {'count': N} for the collection. Useful for debugging.
```

Add a smoke test in intelligence/tests/test_embedder.py:

1. Load 10 chunks from demo_chunks.json
2. Embed them, store_embeddings
3. Query for the first chunk's content (using embed_query – but with chunk content as query for testing, the chunk should retrieve itself)
4. Verify the chunk's own ID is in the top 3 results

Prompt 10 — Bulk-load demo dataset into ChromaDB

PROMPT TO PASTE INTO YOUR LLM

Write intelligence/scripts/load_demo_to_chroma.py:

1. Load demo_chunks.json
2. Create simple Chunk-shaped objects (use dataclass or dict) – note: in real use, Person 1 will pass real SQLAlchemy Chunk objects, but for this script we don't have access to their models, so use a local dataclass that matches the ChunkLike Protocol
3. Embed all chunks: embed_chunks([c['content'] for c in demo_chunks])
4. Store in ChromaDB: store_embeddings(chunks, embeddings)
5. Print collection stats
6. Run a sample query (e.g., 'python async patterns') and print top 5 results

Run this script. The first run will populate ChromaDB. Subsequent runs upsert (no duplicates). This gives you a working corpus for the search module to query against.

Commit checkpoint

git commit -m 'feat(intelligence): BGE embedder + ChromaDB integration'. At Hour 14 you have semantic search working end-to-end at the module level.

7. Hours 14-20 — Hybrid Search (BM25 + Dense + RRF)

Goal: a single `hybrid_search()` function that runs BM25 and dense retrieval in parallel and fuses the results via Reciprocal Rank Fusion. This is the highest-impact change for retrieval quality (+13-26% recall over dense-only per the research).

Prompt 11 — BM25 sparse retriever

PROMPT TO PASTE INTO YOUR LLM

In `intelligence/search.py`, build the BM25 retriever.

Module-level state:

1. `_BM25 = None` # the `BM25Okapi` instance, built lazily
2. `_CHUNK_IDS = []` # parallel list of chunk IDs in the same order as the BM25 index

Functions:

```
def build_bm25_index(chunks: list) -> None
```

Tokenizes chunk contents (simple lower + split, no stemming) and builds `BM25Okapi`.

Stores both the index and chunk IDs at module level.

Should be called once at startup or after bulk ingest.

```
def bm25_search(query: str, top_k: int = 50) -> list[tuple[str, float]]
```

Tokenizes query, calls `_BM25.get_scores(query_tokens)`. Returns `top_k` (`chunk_id`, `score`) tuples sorted by score desc.

Edge case: if `_BM25` is `None`, return `[]` (caller should fall back to dense only).

```
def update_bm25_index(new_chunks: list) -> None
```

`BM25Okapi` doesn't support incremental updates well – easier to just rebuild from scratch. For the hackathon this is fine; we have under 10K chunks. Rebuild from ChromaDB content (use `collection.get()` to retrieve all chunks).

Add a test in `intelligence/tests/test_search.py`:

1. Load demo chunks, build BM25 index
2. Search 'python async patterns' – verify a technical chunk with those keywords ranks high
3. Search 'asynchronous python coroutines' – verify same chunk still ranks (synonyms)
4. Search 'asfdghjkl' – verify empty or low-score results

Prompt 12 — RRF fusion

PROMPT TO PASTE INTO YOUR LLM

In intelligence/search.py, add the RRF fusion function.

```
def reciprocal_rank_fusion(
    *ranked_lists: list[tuple[str, float]], # variable number of (chunk_id, score)
    lists

```

```
k: int = 60
```

```
) -> list[tuple[str, float]]
```

Combines multiple ranked lists into one using RRF. Returns sorted by RRF score desc.

Algorithm (one-line formula from Cormack et al. 2009):

For each chunk_id appearing in any ranked list:

$rrf_score = \sum (1 / (k + rank + 1))$ for each list it appears in, using its rank)
where rank is 0-indexed position in that list.

The k=60 default is the SIGIR 2009 paper's value and what Elasticsearch / Weaviate / Qdrant all use as default.

Add a unit test:

1. Two ranked lists: [('a', 5), ('b', 3), ('c', 1)] and [('b', 0.9), ('c', 0.5), ('a', 0.1)]
2. RRF should rank: 'b' first (consistent middle), 'a' and 'c' close behind
3. Verify the top result is 'b'

Prompt 13 — The hybrid_search public API

PROMPT TO PASTE INTO YOUR LLM

In intelligence/search.py, the main function Person 1 calls:

```
def hybrid_search(
```

```
    query: str,
```

```
    top_k: int = 10,
```

```
    bm25_candidates: int = 50,
```

```
    dense_candidates: int = 50
```

```
) -> list[tuple[str, float]]
```

1. Run BM25 search → top 50 (bm25_candidates) results
2. Run dense search via embedder.query_embeddings → top 50 (dense_candidates) results
3. Fuse via reciprocal_rank_fusion(bm25_results, dense_results, k=60)
4. Return top_k tuples

Edge cases:

- If BM25 index isn't built yet, fall back to dense-only – log a warning
- If query is empty string → return []
- If both retrievers return empty (cold cache) → return []

Add a test that runs a query like 'python async' and prints both individual results and fused results – verify hybrid is sensible.

Also add a quick benchmark:

```
def benchmark_hybrid(n_queries: int = 10):
```

Runs n_queries fixed test queries, prints avg latency.

Target: under 100ms per query for 2,300 chunks.

Prompt 14 — End-to-end integration test

PROMPT TO PASTE INTO YOUR LLM

Create intelligence/tests/test_end_to_end_search.py:

1. Load demo_chunks.json
2. Embed all chunks → store in ChromaDB
3. Build BM25 index from same chunks
4. For 5 hand-picked queries, run hybrid_search and print top 3 results:
 - 'asyncio python' → expect technical chunks about async
 - 'coffee shop' → expect personal chunk about Capitol Hill cafe
 - 'http status codes' → expect reference chunk
 - 'react hooks' → expect technical chunk
 - 'travel plans' → expect personal chunk
5. Eyeball the results – do they make sense? If not, the issue is either:
 - The demo data is too generic (improve templates in demo_data.py)
 - The fusion weights are off (rare with RRF, but possible)
 - BM25 tokenization is too naive (consider adding stopwords)

Paste the output back to me. I'll help diagnose if results look bad.

Commit checkpoint

git commit -m 'feat(intelligence): hybrid BM25 + dense + RRF search'. At Hour 20 you have the most research-validated retrieval stack in any hackathon project at your event.

8. Hours 20-24 — Groq Classifier

Goal: a function that takes chunk texts and returns categories. Called by Person 1's background task after every ingest.

Prompt 15 — Groq client setup

PROMPT TO PASTE INTO YOUR LLM

In intelligence/classifier.py, build the Groq classification module.

Module-level setup:

```
1. import os; from groq import AsyncGroq
2. _CLIENT = AsyncGroq(api_key=os.environ['GROQ_API_KEY'])
3. _MODEL = 'llama-3.3-70b-versatile'
4. _CATEGORIES = ['technical', 'personal', 'reference', 'general']
```

The system prompt template (define as module-level constant):

SYSTEM_PROMPT = '''You are a text classifier. Categorize each text chunk into ONE of exactly four categories:

- technical: code, programming concepts, technical documentation, software architecture
- personal: personal notes, plans, preferences, life events, relationships
- reference: facts, definitions, lookup info, cheat sheets, formulas
- general: meeting notes, miscellaneous thoughts, anything that doesn't clearly fit above

Respond with ONLY a JSON array of category strings, one per chunk, in the same order as the input. No explanation. No other text. Example: ["technical", "personal", "general"]'''

Prompt 16 — Batched classification

PROMPT TO PASTE INTO YOUR LLM

Add the main classify function:

```
async def classify_chunks(
    texts: list[str],
    batch_size: int = 10
) -> list[str]
```

Classifies a list of chunk texts. Returns list of category strings (one per text, in same order). Each must be one of the four valid categories.

Implementation:

1. Process in batches of batch_size (default 10) to keep prompts under context window
2. For each batch, build a user message:
'Classify these chunks:\n\n[1] {first_100_chars_of_text_1}\n\n[2] {text_2}\n...'
3. Call: response = await _CLIENT.chat.completions.create(
model=_MODEL,
messages=[{'role': 'system', 'content': SYSTEM_PROMPT}, {'role': 'user', 'content':
user_msg}],
temperature=0, # deterministic
max_tokens=200, # categories are short
response_format={'type': 'json_object'} # if model supports it
)
4. Parse the response (JSON array). Validate each item is in _CATEGORIES; default to 'general' for invalid responses.
5. Append batch results to the overall list.
6. Add 0.5 second sleep between batches if rate-limited (Groq free tier is ~30 req/min).

Edge cases:

- Empty texts list → return []
- Groq API error (timeout, rate limit) → log error, return ['general'] * len(batch)
- Response is not valid JSON → log, return ['general'] * len(batch)

Add a test:

1. Sample 20 chunks from demo_chunks.json
2. Call await classify_chunks([c['content'] for c in chunks])
3. Print original category vs predicted category
4. Verify accuracy is >70% (demo data is on-topic enough that an LLM should nail it)

Heads-up

Groq's free-tier rate limit is ~30 requests per minute. If you ingest 200 chunks with batch_size=10, that's 20 requests in quick succession — you'll hit the limit. The 0.5s sleep between batches keeps you under it. If you still hit limits during the demo, increase batch_size to 20 (fewer requests, longer prompts) or add exponential backoff.

Prompt 17 — Sync wrapper for Person 1's background task

PROMPT TO PASTE INTO YOUR LLM

Person 1's BackgroundTasks doesn't natively support async functions cleanly. Add a sync wrapper:

```
def classify_chunks_sync(texts: list[str]) -> list[str]:  
    Wraps the async version using asyncio.run(). Use this from Person 1's  
    BackgroundTask.
```

```
import asyncio
```

```
def classify_chunks_sync(texts: list[str]) -> list[str]:  
    return asyncio.run(classify_chunks(texts))
```

Note: asyncio.run creates a new event loop. If called from inside an async context, it errors. Person 1's BackgroundTask runs in its own thread pool, so this is fine.

Commit checkpoint

git commit -m 'feat(intelligence): Groq classifier with batching'

9. Hours 24-30 — Quiz Generator

Goal: a function that takes 5 chunks and returns 5 multiple-choice questions about them, generated by Groq.

Prompt 18 — Quiz prompt design

PROMPT TO PASTE INTO YOUR LLM

In intelligence/quiz.py, build the quiz generator.

Module-level setup:

```
1. import os; from groq import AsyncGroq
2. _CLIENT = AsyncGroq(api_key=os.environ['GROQ_API_KEY'])
3. _MODEL = 'llama-3.3-70b-versatile'
```

The system prompt:

```
QUIZ_SYSTEM_PROMPT = '''You are creating a recall quiz to help someone remember
what they wrote. Given a chunk of text, generate ONE multiple-choice question
that tests whether the reader remembers the key fact or insight in the chunk.
```

Rules:

1. The question must be answerable from the chunk content alone
2. Provide exactly 4 options labeled A, B, C, D
3. Exactly ONE option is correct (must be derivable from the chunk)
4. The other three options must be PLAUSIBLE but WRONG – same domain, similar form, but factually incorrect for this chunk
5. Do not write "According to the chunk..." – write the question directly
6. Keep the question under 25 words

Respond with ONLY a JSON object:

```
{"question\": \"...\", \"options\": [\"A\", \"B\", \"C\", \"D\"],
\"correct_index\": 0-3}'''
```

Prompt 19 — Per-chunk quiz generation

PROMPT TO PASTE INTO YOUR LLM

Add the main function:

```
async def generate_quiz_questions(
    chunks: list, # ChunkLike objects
    target_count: int = 5
) -> list[dict]
Returns a list of question dicts:
[
    {
        'chunk_id': str,
        'question': str,
        'options': list[str] (length 4),
        'correct_index': int (0-3)
    },
    ...
]
```

Implementation:

1. Take `min(target_count, len(chunks))` chunks
2. For each chunk, build a user prompt: `'Chunk:\n{chunk.content}\n\nGenerate one MCQ.'`
3. Call Groq with `QUIZ_SYSTEM_PROMPT`, `temperature=0.3` (slight variety, mostly deterministic), `response_format={'type': 'json_object'}`
4. Parse the JSON response. Validate:
 - 'question' is non-empty string
 - 'options' is list of length 4 with non-empty strings
 - 'correct_index' is int in `[0, 3]`
5. If validation fails, skip that chunk and try the next one. Continue until you have `target_count` questions or run out of chunks.
6. Run all generations in parallel via `asyncio.gather()` – 5 questions in parallel should take 1-2 seconds total instead of 5-10 seconds sequentially.

Edge cases:

- Empty chunks list → return `[]`
- All chunks fail Groq generation → return whatever succeeded (0 questions is acceptable; Person 1's endpoint can decide what to do)
- Groq returns malformed JSON → skip that chunk, log warning

Prompt 20 — Sync wrapper

PROMPT TO PASTE INTO YOUR LLM

Add a sync wrapper just like the classifier:

```
def generate_quiz_questions_sync(chunks: list, target_count: int = 5) ->
list[dict]:
    return asyncio.run(generate_quiz_questions(chunks, target_count))
```

Note: Person 1's `/api/quiz/start` endpoint is async, so they should use the async version directly. The sync wrapper is just in case it's needed in `BackgroundTasks` somewhere.

Prompt 21 — End-to-end quiz test

PROMPT TO PASTE INTO YOUR LLM

Write intelligence/tests/test_quiz.py:

1. Load 5 chunks from demo_chunks.json (mix of categories)
2. Call await generate_quiz_questions(chunks, target_count=5)
3. For each question, print:
 - The chunk content (first 100 chars)
 - The question
 - All 4 options labeled A-D
 - Which is marked correct
4. Eyeball the questions:
 - Is the correct answer actually correct (derivable from the chunk)?
 - Are the wrong answers plausible (not 'the moon')?
 - Is the question reasonable difficulty (not trivial, not impossible)?

Paste output. If quality is bad, we'll iterate on the system prompt.

Heads-up

Quiz quality is a known weak point of LLM generation. Common failure modes: questions too easy (answer literally appears in the question), distractors too obviously wrong, questions about meta-information rather than content ("What does this chunk discuss?" instead of testing actual recall). If you see these, iterate on the system prompt with explicit negative examples ("Do NOT generate questions like X").

Commit checkpoint

git commit -m 'feat(intelligence): Groq quiz generator with parallel inference'. At Hour 30 all six modules are functional.

10. Hours 30-38 — Integration Testing with Person 1

Goal: Person 1's endpoints are calling YOUR modules with REAL data. End-to-end flow works from file upload to dashboard.

Prompt 22 — Pull P1's branch and test integration

PROMPT TO PASTE INTO YOUR LLM

In your local clone:

1. `git checkout feat/intelligence` (your branch)
2. `git fetch origin`
3. `git merge origin/feat/backend` (pull P1's latest)
4. Resolve any merge conflicts (should be minimal – you've worked in different folders)
5. Run Person 1's test: `cd dory-md && uvicorn backend.main:app --reload`
6. In another terminal: `bash backend/scripts/smoke_test.sh` (P1 wrote this)

If smoke test passes, your modules are wired up correctly. If not, common issues:

- Import errors: P1 might be importing `intelligence.foo.bar` but your function is in `baz`
- Signature mismatch: P1 might call `compute_retention(chunks, time_offset=0)` but your sig is `(chunks, time_offset_hours=0)`
- Async/sync mismatch: P1 calls `await classify_chunks()` but you only have sync wrapper

Fix forward – work with P1 in chat to align contracts. Your contract is more stable than their endpoint code, so usually they adjust.

Prompt 23 — Load demo dataset into the live system

PROMPT TO PASTE INTO YOUR LLM

Write `intelligence/scripts/seed_demo_database.py`:

This script loads `demo_chunks.json` directly into Person 1's SQLite database AND ChromaDB. It bypasses the `/api/ingest` endpoint because that endpoint expects file uploads.

1. Import P1's models: `from backend.models.chunk import Document, Chunk`
2. Import P1's db setup: `from backend.models import async_session_maker, init_db`
3. Load `demo_chunks.json`
4. Group chunks by `source_name`, create one `Document` per source
5. Insert Documents and Chunks into SQLite (use P1's `create_chunks_batch` where possible)
6. Build `Chunk` objects matching `ChunkLike` for your modules
7. Embed all chunks: `embed_chunks([c.content for c in chunks])`
8. Store in ChromaDB: `store_embeddings(chunks, embeddings)`
9. Build BM25 index: `build_bm25_index(chunks)`
10. Run classifier on all chunks, update categories in SQLite
11. Print: `'Loaded 90 chunks across 4 categories. Demo ready.'`

Run this script. Verify by hitting `http://localhost:8000/api/health` and seeing realistic category counts.

Prompt 24 — Test all integration points

PROMPT TO PASTE INTO YOUR LLM

Run a manual test plan:

1. Server up, demo data loaded.
2. `<code>curl http://localhost:8000/api/health</code>` → should show 4 categories with realistic retention values
3. `<code>curl 'http://localhost:8000/api/health?time_offset_hours=720'</code>` (one month into the future) → retentions should be visibly lower
4. `<code>curl -X POST http://localhost:8000/api/search -H 'Content-Type: application/json' -d '{"query": "python async", "top_k": 5}'</code>` → should return technical chunks about async
5. `<code>curl http://localhost:8000/api/discovery</code>` → may return `has_discovery=true` or `false` depending on your data
6. `<code>curl -X POST http://localhost:8000/api/quiz/start -H 'Content-Type: application/json' -d '{}</code>` → should return 5 questions about your lowest-retention chunks

If any of these fails, debug systematically:

- Look at FastAPI logs for the error
- Add `print()` statements in your intelligence module to see what data it's receiving
- Verify ChromaDB has data: `get_collection_stats()`

Don't fix bugs in P1's code – open a GitHub issue or ping in chat. Your job is your modules, theirs is the API layer.

Prompt 25 — Performance profiling

PROMPT TO PASTE INTO YOUR LLM

Run a load test on `/api/health` (the most-called endpoint, hit on every Time Machine slider drag):

```
for i in {1..100}; do curl -s -o /dev/null -w '%{time_total}\n'
'http://localhost:8000/api/health?time_offset_hours=$RANDOM' & done; wait
```

Calculate p50, p95, p99 from the output (sort, then take the 50th, 95th, 99th values).

Targets:

- p50 < 50ms (fast enough for slider drag)
- p95 < 100ms (no visible lag)
- p99 < 200ms (occasional outliers OK)

If you exceed targets, the bottleneck is likely:

- Async DB query taking too long (P1's problem, but you can suggest indexes)
- Decay computation not vectorized (your problem)
- JSON serialization (use `ORJSONResponse` – P1 should already have)

Profile with `cProfile` or just print timings inside `compute_retention` to localize.

11. Hours 38-48 — Polish, Bug Bash, Demo Prep

Hour 38-42: bug bash with the team

Sit with Persons 1 and 3, click through every flow on the deployed app. Find bugs in:

- Edge cases in retention math (chunks with future timestamps, NaN values)
- Search results that look wrong for sample queries
- Quiz questions that are too easy or nonsensical
- Classifier mislabeling personal-domain chunks as technical or vice versa

Prompt 26 — Tune classifier accuracy

PROMPT TO PASTE INTO YOUR LLM

If the classifier is mislabeling things during the bug bash, iterate on the system prompt:

1. Collect 5-10 examples where it got it wrong
2. Look for patterns — does it default to 'general' too often? Confuse 'reference' with 'technical'?
3. Update SYSTEM_PROMPT to add explicit examples of the borderline cases
4. Consider adding a few-shot example at the end of the system prompt:
'Examples:\n - "The HTTP 401 status code means Unauthorized" → reference\n - "Notes from 1:1 with manager about Q4 planning" → personal\n - "TypeScript generics let you write reusable functions" → technical\n - "Some thoughts on the article I read this morning" → general'
5. Re-run the classifier on the same 20 demo chunks. Verify accuracy improved.

Prompt 27 — Tune quiz quality

PROMPT TO PASTE INTO YOUR LLM

If quiz questions are weak (too easy, distractors implausible), iterate on QUIZ_SYSTEM_PROMPT:

1. Collect 3-5 weak questions
2. Add explicit negative examples to the prompt:
'BAD example: "What does the chunk discuss? A) The chunk discusses async B) Pizza C) Mountain biking D) Quantum physics" — too obvious
GOOD example: "In Python asyncio, what does asyncio.gather() return when one coroutine raises? A) It cancels all others by default B) It collects results then raises C) It returns None D) It silences the exception"'
3. Bump temperature slightly (0.3 → 0.5) for more variety in distractors
4. Re-run on 5 sample chunks. Verify questions improved.

Hour 42-46: dry-run the demo

With your team, run a 5-minute mock demo. Stand up, click through, narrate. Time it. Find the parts where the system feels slow or breaks. The Time Machine slider is your moment — make sure it's perceptibly responsive.

Hour 46-48: buffer

Don't add features. Sleep, eat, prep mentally. If the demo works at Hour 46, sit on your hands for the last 2 hours.

12. Stretch Tasks (if you finish early)

If you're done by Hour 30, here are extras ranked by demo impact:

- **Implement FSRS-inspired stability updates** — instead of $S *= 1.5$ on review, use the FSRS formula based on retrievability at review time. Real research-grade decay.
- **Add ColBERT-style re-ranking on top 20 results** — install fastembed-colbert, rerank after RRF. The literature says this pushes recall further.
- **Cross-modal search (text query → code chunks)** — useful if you have many code-heavy notes. Doesn't need new infrastructure.
- **Decay-based discovery prioritization** — instead of just lowest retention, weight by category recency and recent search history.
- **Parallelize ChromaDB store_embeddings** — for very large ingests. Probably not needed for the demo.

13. Descope Ladder (if you fall behind)

Cut features in this order — top of list goes FIRST:

#	Cut	What you keep	Demo impact
1	Quiz mode entirely	All other modules	One less feature shown — fine
2	Groq classifier	Hardcode all chunks to 'general'	Dashboard has one big bar instead of 4 — less visual but works
3	Hybrid search (BM25)	Dense-only search	Search a bit weaker but still semantic
4	ChromaDB persistence	In-memory NumPy similarity	Slower at scale but zero deps
5	Backdated demo dataset	Just live-uploaded data during demo	Time Machine slider less impressive but still works

14. Handoff Checklists

Hour 4 → tell P1: 'Demo dataset ready, decay sig confirmed'

- Share demo_chunks.json so P1 can use it for testing too
- Confirm signature: compute_retention(chunks, time_offset_hours=0)
- Push your branch with placeholder stubs so P1 can import without errors

Hour 14 → tell P1: 'Embedder + ChromaDB are live'

- Share location of chroma_db/ folder (gitignored, but they need to know it exists)
- Confirm functions: embed_chunks, store_embeddings
- They can now call from /api/ingest's BackgroundTask

Hour 20 → tell P1: 'Hybrid search ready'

- Confirm: hybrid_search(query: str, top_k: int = 10)
- They can wire /api/search to this
- Note: BM25 index needs to be built once on startup — see your build_bm25_index function

Hour 30 → 'All modules ready, let's wire it up'

- Pull P1's branch, run their smoke test, fix integration bugs
- Run seed_demo_database.py to populate live system

15. Troubleshooting Guide

Symptom	Likely cause	Fix
First call to SentenceTransformer hangs	Model not pre-downloaded, slow WiFi at venue	Use phone hotspot, or download from a friend's laptop and copy ~/.cache/huggingface
ImportError: No module named sentence_transformers	venv not activated or package not installed	source .venv/bin/activate && pip install -r requirements.txt
ChromaDB error: 'Collection already exists with different settings'	Tried to recreate collection with different M or ef_construction	Delete ./chroma_db folder, restart. M and ef_construction are FROZEN after creation.
compute_retention returns NaN values	Some chunk has invalid datetime or zero stability_S	Add validation: clamp S to minimum 0.1, check last_accessed is not None
Hybrid search is slow (> 200ms)	BM25 index rebuilt on every query, or ChromaDB query slow	Build BM25 once at startup; verify ChromaDB collection has the HNSW config
Groq returns 429 Too Many Requests	Rate limit on free tier (~30 req/min)	Increase batch_size, add longer sleep between batches, retry with exponential backoff
Groq returns malformed JSON	Model occasionally violates the JSON-only instruction	Catch the parse error, log, return fallback (categories: 'general'; quiz: skip chunk)
BM25 returns empty for all queries	Index not built, or tokenizer too aggressive (filtering everything)	Add print() inside build_bm25_index to verify it ran; check tokens are non-empty
Classifier mislabels obvious cases	System prompt too vague	Add 4-shot examples to the prompt, one per category
Quiz questions are too easy	Temperature too low or instructions too vague	Bump temperature to 0.5, add "BAD example" in system prompt as anti-example
Module import works in tests but fails in P1's server	Different working directory or PYTHONPATH	Verify intelligence/ has __init__.py at every level; run server from project root
aggregate_by_category produces wrong urgency labels	Threshold logic inverted (high retention = low urgency)	Reread the spec: high retention means user remembers it = LOW urgency to review

If something is wrong or unclear, ask in team chat. Don't suffer alone.